

RAG — Retrieval Augmented Generation

The Complete Zero-to-Hero Guide

Ingestion · Embeddings · Vector Databases · Retrieval · Generation · Implementation · Interview Prep

Written in simple English, with a MERN / Node.js developer in mind

Table of Contents

1. What is RAG and why do we need it
2. The Big Picture: RAG Architecture
3. Phase 1 — Data Ingestion Pipeline (Loading, Chunking, Metadata)
4. Embeddings — Turning Text into Numbers
5. Vector Databases — Deep Dive
6. Phase 2 — Retrieval (Search Strategies)
7. Phase 3 — Generation / Augmentation
8. Full End-to-End Implementation (MERN Stack)
9. Advanced RAG Concepts
10. RAG vs Fine-tuning vs Long Context
11. Common Mistakes and How to Avoid Them
12. Interview Questions (Basic to Advanced)
13. Final Cheat Sheet

1. What is RAG and Why Do We Need It

1.1 The Problem with a Normal LLM

An LLM like GPT, Claude, or Gemini is trained once, on data up to a certain date. After training, it is frozen. This causes three big problems:

- **Knowledge cutoff:** It does not know about events after its training date.
- **No private data:** It has never seen your company's internal documents, your database, or your PDFs.
- **Hallucination:** When the model does not know an answer, it sometimes makes up a confident-sounding but wrong answer.

Analogy: A normal LLM is like a brilliant student who took a closed-book exam. They memorized a huge amount during study (training), but in the exam room they cannot open a book to check facts. If they forget something, they may guess — and guess wrong.

1.2 What RAG Does

RAG (Retrieval Augmented Generation) turns the closed-book exam into an open-book exam. Before the LLM answers a question, we:

1. Search our own documents for the most relevant pieces of information.
2. Hand those pieces to the LLM as "context" along with the user's question.
3. The LLM reads the context and writes an answer grounded in that context.

One-line definition: RAG = Search (Retrieval) + Answer Generation, where the search results become the LLM's reference material for that one answer.

1.3 Why RAG Instead of Just Fine-tuning the Model

Need	Better choice	Why
Add new facts daily (prices, tickets, docs)	RAG	Just update the database, no retraining
Teach a new writing style or tone	Fine-tuning	This is about behavior, not facts
Need to cite sources / be auditable	RAG	You always know which document was used
Very large, constantly changing knowledge base	RAG	Cheaper than retraining a model every day

1.4 Real World Use Cases

- Customer support chatbot that answers from your company's help docs
- "Chat with your PDF" apps
- Internal company knowledge assistant (HR policy, code documentation)
- AI search engines that show sources (like Perplexity)
- Legal/medical assistants that must quote real documents, not guesses

2. The Big Picture: RAG Architecture

RAG has exactly **two separate phases**. Most confusion in RAG comes from mixing these two up, so keep them clearly separate in your head.

2.1 Phase A — Indexing (done once, offline, ahead of time)

```
Raw Documents (PDF, Word, Web pages, DB rows)
  |
  v
Load / Parse text
  |
  v
Split into small Chunks
  |
  v
Convert each chunk into an Embedding (vector)
  |
  v
Store [chunk text + embedding + metadata] in a Vector Database
```

2.2 Phase B — Retrieval + Generation (happens every time a user asks a question)

```
User Question
  |
  v
Convert question into an Embedding (SAME embedding model as above)
  |
  v
Search Vector DB for the most similar chunks (top-k)
  |
  v
Build a prompt = (retrieved chunks) + (user question)
  |
  v
Send prompt to the LLM
  |
  v
LLM generates the final answer, grounded in the retrieved chunks
```

Analogy: Phase A is like organizing a library — you take every book, cut out the important paragraphs, and file them by topic in a smart filing cabinet. Phase B is like a librarian who, when you ask a question, instantly pulls the 3-4 most relevant paragraphs from the cabinet and hands them to an expert who writes you a short answer using only those paragraphs.

Phase A runs rarely (when you add or update documents). Phase B runs on every single chat message. This is why ingestion can be slow, but retrieval must be fast (milliseconds).

3. Phase 1 — Data Ingestion Pipeline

3.1 Step 1: Loading Documents

Before anything else, you need raw text out of whatever format your data is in.

Source	Node.js tool
PDF	<code>pdf-parse</code> , <code>pdfjs-dist</code>
Word (.docx)	<code>mammoth</code>
Web pages	<code>cheerio</code> + <code>axios</code> , or <code>puppeteer</code> for JS-heavy sites
CSV / Excel	<code>papaparse</code> , <code>xlsx</code>
Database (MongoDB)	Just query and concatenate the text fields
Markdown	plain <code>fs.readFile</code> , it's already text

Implementation: Loading a PDF in Node.js

```
const fs = require("fs");
const pdfParse = require("pdf-parse");

async function loadPDF(filePath) {
  const buffer = fs.readFileSync(filePath);
  const data = await pdfParse(buffer);
  return data.text; // full raw text of the PDF
}
```

Implementation: Loading a Web Page

```
const axios = require("axios");
const cheerio = require("cheerio");

async function loadWebPage(url) {
  const { data: html } = await axios.get(url);
  const $ = cheerio.load(html);
  $("script, style, nav, footer").remove(); // remove noise
  return $("body").text().replace(/\s+/g, " ").trim();
}
```

3.2 Step 2: Chunking (Text Splitting)

Why we cannot feed the whole document at once

- LLMs have a maximum context window (limited number of tokens).
- Embedding models also have a token limit per input.
- If a chunk is too big, it mixes many topics together, so the embedding becomes "blurry" and matches badly.
- If a chunk is too small, it loses surrounding context and meaning.

Analogy: Imagine highlighting an entire textbook page as "important" versus highlighting just the one sentence that answers the question. A whole page is too vague to be useful; one isolated word is too little. Chunking finds the right-sized highlight — usually a paragraph or two.

Key terms

- **Chunk size:** how many characters/tokens go in one chunk (common: 300-1000 tokens).

- **Chunk overlap:** a small repeated slice between consecutive chunks (common: 10-20% of chunk size) so a sentence cut in half at the boundary is not lost.

Chunking Strategies

Strategy	How it works	Best for
Fixed-size	Cut every N characters, blindly	Quick prototypes only
Recursive character splitting	Try to split on paragraph, then sentence, then word, in order, until chunk fits the size	General purpose — most popular default
Sentence-based	Never break in the middle of a sentence	Q&A style content
Document-structure-aware	Split on Markdown headers (#, ##) or HTML tags	Technical docs, wikis
Semantic chunking	Split where the topic/meaning actually changes (using embeddings of sentences)	Long unstructured text, books
Token-based	Split by actual model tokens, not characters	When you must respect a hard token budget

Implementation: Recursive Character Splitter (from scratch)

```
function splitText(text, chunkSize = 800, overlap = 150) {
  const separators = ["\n\n", "\n", ". ", " "]; // big breaks first
  function splitBySeparators(str, seps) {
    if (str.length <= chunkSize) return [str];
    const sep = seps[0] || "";
    const parts = sep ? str.split(sep) : str.split("");
    let chunks = [];
    let current = "";
    for (const part of parts) {
      const candidate = current ? current + sep + part : part;
      if (candidate.length > chunkSize && current) {
        chunks.push(current);
        current = part;
      } else {
        current = candidate;
      }
    }
    if (current) chunks.push(current);
    // if any chunk is still too big, split it further with next separator
    return chunks.flatMap(c =>
      c.length > chunkSize && seps.length > 1 ? splitBySeparators(c, seps.slice(1)) : [c]
    );
  }

  const rawChunks = splitBySeparators(text, separators);

  // add overlap between consecutive chunks
  const finalChunks = [];
  for (let i = 0; i < rawChunks.length; i++) {
    const prevTail = i > 0 ? rawChunks[i - 1].slice(-overlap) : "";
    finalChunks.push((prevTail + rawChunks[i]).trim());
  }
  return finalChunks;
}
```

In real projects, you do not have to write this by hand. **LangChain.js** gives you this for free:

```
const { RecursiveCharacterTextSplitter } = require("langchain/text_splitter");

const splitter = new RecursiveCharacterTextSplitter({
  chunkSize: 800,
```

```
    chunkOverlap: 150,  
  });  
  
  const chunks = await splitter.splitText(rawText);
```

3.3 Step 3: Attaching Metadata

Every chunk should be stored with extra information, not just the raw text. This metadata is used later for filtering and for showing "sources" to the user.

```
{  
  text: "Affectra uses MediaPipe for real-time facial emotion detection...",  
  metadata: {  
    source: "affectra-docs.pdf",  
    page: 4,  
    chunkIndex: 12,  
    createdAt: "2026-06-20"  
  }  
}
```

Common mistake: Forgetting metadata at ingestion time means you cannot later tell the user "this answer came from page 4 of the HR policy PDF." Always plan metadata before you start ingesting, not after.

4. Embeddings — Turning Text into Numbers

4.1 What is an Embedding

An embedding is a list of numbers (a vector) that represents the **meaning** of a piece of text. A special neural network (the embedding model) reads the text and outputs this vector. Texts with similar meaning get vectors that are close to each other in this number-space, even if they use completely different words.

Analogy: Think of embeddings as GPS coordinates, but for meaning instead of location. "I love dogs" and "I adore puppies" land very close together on this meaning-map, even though they share almost no words. "I love dogs" and "The stock market crashed" land far apart.

4.2 What Does the Vector Look Like

```
"I love programming in JavaScript"
|
v (embedding model)
v
[0.0123, -0.045, 0.221, ..., 0.018] // e.g. 1536 numbers (dimensions)
```

4.3 Popular Embedding Models

Model	Provider	Dimensions	Notes
text-embedding-3-small	OpenAI	1536	Cheap, very popular, paid API
text-embedding-3-large	OpenAI	3072	Higher accuracy, costs more
embed-english-v3	Cohere	1024	Good for search-focused use cases
all-MiniLM-L6-v2	Sentence-Transformers (open source)	384	Free, runs locally, smaller and faster
text-embedding-004	Google Gemini	768	Free tier available

Golden rule: You must use the exact same embedding model to embed your documents (at ingestion time) and to embed the user's question (at retrieval time). Mixing two different embedding models gives you garbage search results, because the two sets of vectors are not on the same "meaning map."

4.4 Implementation: Generating an Embedding (Node.js + OpenAI)

```
const OpenAI = require("openai");
const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

async function getEmbedding(text) {
  const response = await openai.embeddings.create({
    model: "text-embedding-3-small",
    input: text,
  });
  return response.data[0].embedding; // array of numbers
}
```

Embedding many chunks at once (batching)

```
async function embedChunks(chunks) {
  const response = await openai.embeddings.create({
    model: "text-embedding-3-small",
```

```

    input: chunks, // array of strings, sent together = faster & cheaper
  });
  return response.data.map(d => d.embedding);
}

```

4.5 Measuring Similarity Between Two Vectors

Once we have vectors, we need a formula to ask: "how close are these two vectors?" Three common formulas:

Method	Idea	Range
Cosine similarity	Measures the <i>angle</i> between two vectors, ignoring their length. Most popular for text.	-1 to 1 (1 = identical meaning direction)
Euclidean distance	Straight-line distance between two points	0 to infinity (0 = identical)
Dot product	Multiply and sum each pair of numbers; combines angle and length	any number

Cosine similarity formula in plain JavaScript

```

function cosineSimilarity(vecA, vecB) {
  let dot = 0, normA = 0, normB = 0;
  for (let i = 0; i < vecA.length; i++) {
    dot += vecA[i] * vecB[i];
    normA += vecA[i] * vecA[i];
    normB += vecB[i] * vecB[i];
  }
  return dot / (Math.sqrt(normA) * Math.sqrt(normB));
}

```

Analogy: Imagine two arrows starting from the same point. Cosine similarity checks if they point in roughly the same direction (the angle), no matter how long each arrow is. If two arrows point in almost the same direction, the texts they represent mean almost the same thing.

5. Vector Databases — Deep Dive

5.1 Why a Normal Database is Not Enough

MongoDB, MySQL, PostgreSQL are built to find *exact* matches or ranges (`WHERE age > 20` , `WHERE name = "Anand"`). But "find the chunks whose meaning is closest to this question" is not an exact match problem — it needs to compare millions of vectors and find the nearest ones, fast. That specialized job is what a **vector database** is built for.

Good news for MERN developers: you do **not** have to learn a brand-new database. **MongoDB Atlas** now has built-in **Vector Search**, so you can store vectors in the same MongoDB collection you already use, right next to your normal fields.

5.2 How a Vector DB Actually Searches Fast

Comparing the question's vector against every single stored vector one by one (called a "brute-force" or "k-NN" search) works, but becomes slow once you have millions of vectors. Vector databases use **ANN (Approximate Nearest Neighbor)** algorithms to find "good enough" closest matches almost instantly.

Index type	Idea in simple words
HNSW (Hierarchical Navigable Small World)	Builds a multi-layer "highway map" between vectors. Search starts on a sparse top layer (fast, rough jumps) and zooms into denser lower layers (fine-grained) — like zooming from a country map down to a street map.
IVF (Inverted File Index)	Groups vectors into clusters first. A search only checks the few clusters closest to the query, instead of everything.
LSH (Locality Sensitive Hashing)	Uses hash functions designed so that similar vectors are likely to land in the same hash bucket.

Analogy for HNSW: Finding a friend's house in a new city. You don't walk every street (brute force). You first look at a highway map to get to the right neighborhood (top layer), then a local map to find the right street (middle layer), then walk to the exact house (bottom layer). Each layer narrows the search.

5.3 Popular Vector Databases Compared

Database	Type	Good for
MongoDB Atlas Vector Search	Managed, added on top of MongoDB	MERN projects — keep everything in one database
Pinecone	Fully managed, cloud-only	Production apps, zero infra management
Chroma	Open-source, runs locally or self-hosted	Local development, learning, small projects
Qdrant	Open-source, self-host or managed cloud	High performance, filtering-heavy use cases
Weaviate	Open-source, self-host or managed	Built-in hybrid search support
pgvector	PostgreSQL extension	Teams already using Postgres
Redis (with vector module)	In-memory	Very low latency caching + search

5.4 Implementation: Storing Vectors with MongoDB Atlas Vector Search

Step A — Document shape in MongoDB

```
// collection: "document_chunks"
```

```
{
  _id: ObjectId("..."),
  text: "Affectra detects emotion in real time using MediaPipe...",
  embedding: [0.0123, -0.045, 0.221, /* ...1536 numbers */],
  metadata: { source: "affectra-docs.pdf", page: 4 }
}
```

Step B — Create a Vector Search Index (done once, in Atlas UI or via API)

```
{
  "fields": [
    {
      "type": "vector",
      "path": "embedding",
      "numDimensions": 1536,
      "similarity": "cosine"
    },
    { "type": "filter", "path": "metadata.source" }
  ]
}
```

Step C — Insert chunks (Node.js, using the official driver or Mongoose)

```
const { MongoClient } = require("mongodb");
const client = new MongoClient(process.env.MONGO_URI);

async function storeChunks(chunksWithEmbeddings) {
  const db = client.db("ragApp");
  const collection = db.collection("document_chunks");
  await collection.insertMany(chunksWithEmbeddings);
}

// chunksWithEmbeddings = [{ text, embedding, metadata }, ...]
```

Step D — Search (this is the actual retrieval query)

```
async function vectorSearch(queryEmbedding, topK = 5) {
  const db = client.db("ragApp");
  const collection = db.collection("document_chunks");

  const results = await collection.aggregate([
    {
      $vectorSearch: {
        index: "vector_index",
        path: "embedding",
        queryVector: queryEmbedding,
        numCandidates: 100,
        limit: topK,
      },
    },
    {
      $project: {
        text: 1,
        metadata: 1,
        score: { $meta: "vectorSearchScore" },
      },
    },
  ]).toArray();

  return results;
}
```

5.5 Implementation: Same Thing with Pinecone (alternative)

```
const { Pinecone } = require("@pinecone-database/pinecone");
```

```
const pinecone = new Pinecone({ apiKey: process.env.PINECONE_API_KEY });
const index = pinecone.index("rag-index");

// Insert
await index.upsert([
  { id: "chunk-1", values: embeddingArray, metadata: { source: "affectra-docs.pdf" } },
]);

// Search
const results = await index.query({
  vector: queryEmbedding,
  topK: 5,
  includeMetadata: true,
});
```

6. Phase 2 — Retrieval (Search Strategies)

6.1 The Basic Retrieval Flow

```
1. Take user's question
2. Embed it with the SAME model used during ingestion
3. Run vectorSearch(queryEmbedding, topK)
4. Get back top-k chunks (text + metadata + score)
```

6.2 Simple Similarity Search (the default)

Just return the top-k closest chunks by cosine similarity. Easy, but it has a weakness: it can return chunks that are *all saying almost the same thing*, wasting your top-k slots on repeated information.

6.3 Hybrid Search (Vector + Keyword)

Pure vector search sometimes misses an exact keyword, like a product code "SKU-4521" or a person's name, because embeddings care about meaning, not exact spelling. **Hybrid search** combines:

- **Vector search** (meaning-based, good for "what is similar to this idea")
- **Keyword search / BM25** (exact term-based, good for codes, names, rare words)

The two result lists are merged using a scoring formula (often called **RRF — Reciprocal Rank Fusion**).

Analogy: Vector search is like asking a friend "find me something about happy moments." Keyword search is like asking "find the page with the exact word 'INV-2024-0098'." Hybrid search asks both friends and combines their answers.

6.4 MMR — Maximal Marginal Relevance

MMR picks chunks that are relevant to the question **but also different from each other**, so your top-k results cover more ground instead of repeating the same point five times.

```
// Simplified idea, not exact production code
function mmrSelect(candidates, queryVec, k, lambda = 0.7) {
  const selected = [];
  while (selected.length < k && candidates.length) {
    let best, bestScore = -Infinity;
    for (const c of candidates) {
      const relevance = cosineSimilarity(c.embedding, queryVec);
      const redundancy = selected.length
        ? Math.max(...selected.map(s => cosineSimilarity(s.embedding, c.embedding)))
        : 0;
      const score = lambda * relevance - (1 - lambda) * redundancy;
      if (score > bestScore) { bestScore = score; best = c; }
    }
    selected.push(best);
    candidates = candidates.filter(c => c !== best);
  }
  return selected;
}
```

6.5 Re-ranking

Vector search is fast but a bit rough. A common trick: first retrieve a larger set (say top 25) with the fast vector search, then run a smaller, smarter, slower model (a **cross-encoder re-ranker**, e.g. Cohere Rerank) on just those 25 to pick the best 5. This two-stage approach gets both speed and accuracy.

```
const { CohereClient } = require("cohere-ai");
const cohere = new CohereClient({ token: process.env.COHERE_API_KEY });

async function rerank(query, documents, topN = 5) {
  const response = await cohere.rerank({
    query,
    documents, // array of chunk texts
    topN,
    model: "rerank-english-v3.0",
  });
  return response.results; // sorted, most relevant first
}
```

6.6 Metadata Filtering

You can narrow the search before or during the vector search using metadata, e.g. "only search chunks where `source = 'hr-policy.pdf'`" or "only chunks created in the last 30 days." This avoids irrelevant documents polluting the results.

6.7 Multi-Query Retrieval

Sometimes one user question is ambiguous or too narrow. The LLM is asked to rewrite it into 3-4 different phrasings, each is searched separately, and the results are merged. This catches relevant chunks that the original single phrasing would have missed.

6.8 HyDE — Hypothetical Document Embeddings

Instead of embedding the short user question directly, first ask the LLM to write a hypothetical, ideal answer to the question (even if it might be wrong), then embed *that* hypothetical answer and search with it. A full hypothetical answer often matches the style and vocabulary of real document chunks better than a short question does.

6.9 Parent-Document / Small-to-Big Retrieval

Store small chunks for accurate searching, but when one is matched, return its larger parent section (e.g. the whole page) to the LLM. This gives precise search with rich context for generation.

Summary table of retrieval upgrades:

Problem	Fix
Missing exact keywords/codes	Hybrid search
Repeated/redundant chunks	MMR
Rough ranking from fast vector search	Re-ranking
Wrong documents mixed in	Metadata filtering
Question phrased ambiguously	Multi-query retrieval
Question vocabulary differs from document style	HyDE
Chunk too small to give full context	Parent-document retrieval

7. Phase 3 — Generation (Augmentation)

7.1 Building the Final Prompt

This is the "Augmented" part of RAG: we inject the retrieved chunks into the prompt we send to the LLM.

```
function buildPrompt(question, retrievedChunks) {
  const context = retrievedChunks
    .map((c, i) => `[Source ${i + 1}]: ${c.text}`)
    .join("\n\n");

  return `You are a helpful assistant. Answer the question using ONLY the
context below. If the answer is not in the context, say
"I don't have enough information to answer that."

Context:
${context}

Question: ${question}

Answer:`;
}
```

7.2 Why the System Prompt Matters So Much

- Tell the model to **only** use the given context — this is the #1 defense against hallucination.
- Tell it what to say when the answer isn't in the context, instead of guessing.
- Ask it to mention which source number it used, so you can show citations.

7.3 Implementation: Calling the LLM with Context

```
const OpenAI = require("openai");
const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

async function generateAnswer(question, retrievedChunks) {
  const prompt = buildPrompt(question, retrievedChunks);

  const completion = await openai.chat.completions.create({
    model: "gpt-4o-mini",
    messages: [
      { role: "system", content: "You answer strictly from the given context." },
      { role: "user", content: prompt },
    ],
    temperature: 0.2, // low temperature = more factual, less creative
  });

  return completion.choices[0].message.content;
}
```

Why temperature is low in RAG: A high temperature makes the model more "creative," which is exactly what you do NOT want when the goal is faithfulness to real documents.

7.4 Streaming the Answer (better UX)

```
const stream = await openai.chat.completions.create({
  model: "gpt-4o-mini",
  messages: [...],
  stream: true,
});
```

```
for await (const chunk of stream) {
  const token = chunk.choices[0]?.delta?.content || "";
  res.write(token); // pipe straight to the Express response
}
```

7.5 Returning Citations to the Frontend

```
res.json({
  answer: generatedText,
  sources: retrievedChunks.map(c => ({
    source: c.metadata.source,
    page: c.metadata.page,
  })),
});
```

8. Full End-to-End Implementation (MERN Stack)

8.1 Folder Structure

```
rag-app/
├── server/
│   ├── ingest.js          // run once to load + chunk + embed + store docs
│   ├── routes/chat.js     // Express route: question in, answer out
│   ├── utils/embed.js
│   ├── utils/vectorStore.js
│   ├── utils/chunk.js
│   ├── server.js
│   └── .env
└── client/                // React app
    └── src/components/ChatBox.jsx
```

8.2 Packages You Need

```
npm install express mongodb openai dotenv pdf-parse cors
# optional helpers:
npm install langchain @langchain/openai @langchain/mongodb cohere-ai
```

8.3 ingest.js (Phase A — run this once per document)

```
require("dotenv").config();
const { loadPDF } = require("../utils/loaders");
const { splitText } = require("../utils/chunk");
const { embedChunks } = require("../utils/embed");
const { storeChunks } = require("../utils/vectorStore");

async function ingest(filePath, sourceName) {
  const rawText = await loadPDF(filePath);
  const chunks = splitText(rawText, 800, 150);
  const embeddings = await embedChunks(chunks);

  const docs = chunks.map((text, i) => ({
    text,
    embedding: embeddings[i],
    metadata: { source: sourceName, chunkIndex: i },
  }));

  await storeChunks(docs);
  console.log(`Ingested ${docs.length} chunks from ${sourceName}`);
}

ingest("../docs/affectra-handbook.pdf", "affectra-handbook.pdf");
```

8.4 routes/chat.js (Phase B — Express endpoint)

```
const express = require("express");
const router = express.Router();
const { getEmbedding } = require("../utils/embed");
const { vectorSearch } = require("../utils/vectorStore");
const { generateAnswer } = require("../utils/generate");

router.post("/chat", async (req, res) => {
  try {
    const { question } = req.body;

    const queryEmbedding = await getEmbedding(question);
```

```

const topChunks = await vectorSearch(queryEmbedding, 5);
const answer = await generateAnswer(question, topChunks);

res.json({
  answer,
  sources: topChunks.map(c => c.metadata),
});
} catch (err) {
  console.error(err);
  res.status(500).json({ error: "Something went wrong" });
}
});

module.exports = router;

```

8.5 server.js

```

require("dotenv").config();
const express = require("express");
const cors = require("cors");
const chatRoute = require("./routes/chat");

const app = express();
app.use(cors());
app.use(express.json());
app.use("/api", chatRoute);

app.listen(5000, () => console.log("RAG server running on port 5000"));

```

8.6 React Frontend (ChatBox.jsx)

```

import { useState } from "react";

export default function ChatBox() {
  const [question, setQuestion] = useState("");
  const [answer, setAnswer] = useState("");
  const [sources, setSources] = useState([]);
  const [loading, setLoading] = useState(false);

  const askQuestion = async () => {
    setLoading(true);
    const res = await fetch("http://localhost:5000/api/chat", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ question }),
    });
    const data = await res.json();
    setAnswer(data.answer);
    setSources(data.sources);
    setLoading(false);
  };

  return (
    <div>
      <input type="text" value={question} onChange={e => setQuestion(e.target.value)} />
      <div>
        {loading ? "Thinking..." : "Ask"}
      </div>
      <div>
        {answer}
      </div>
      <div>
        {sources.map((s, i) =>
          <div>
            • {s.source} (chunk {s.chunkIndex})
          </div>
        )}
      </div>
    </div>
  );
}

```

```
    })
```

```
    );  
  }
```

8.7 Should You Use LangChain.js Instead of Writing This by Hand

LangChain.js wraps loaders, splitters, embeddings, vector stores, and prompt building into ready-made building blocks. It saves time in real projects. Writing it by hand (like above) is still very valuable while learning, because it forces you to actually understand every step — which matters a lot in interviews.

9. Advanced RAG Concepts

9.1 Agentic RAG

Instead of always doing one fixed retrieve-then-generate pass, an "agent" (the LLM acting in a loop) decides: should I search again with a different query? Should I use a calculator tool? Should I call an API? It keeps reasoning and retrieving in multiple steps until it is confident in the answer.

9.2 Self-RAG / Corrective RAG (CRAG)

The model checks its own retrieved chunks for relevance before answering. If the chunks look weak or off-topic, it can trigger a fallback — like doing a fresh web search, rewriting the query, or simply saying it doesn't have enough information — instead of generating a shaky answer anyway.

9.3 GraphRAG

Instead of (or in addition to) a vector database, build a knowledge graph (entities and their relationships) from your documents. Retrieval then walks the graph to find connected facts, which is powerful for questions involving multi-hop reasoning, like "Which projects use a tool that was also used in another project led by the same person?"

9.4 Query Routing

In a system with multiple data sources (say, one vector DB for HR docs, one for product docs, plus a SQL database), a "router" step first decides which source(s) are relevant to this specific question, then only queries those.

9.5 Semantic Caching

If a new question's embedding is extremely close to a question asked recently, skip the whole retrieval+generation pipeline and return the cached previous answer. This saves cost and latency for common, repeated questions.

9.6 The "Lost in the Middle" Problem

Research shows LLMs pay more attention to information at the very start and very end of a long prompt, and can under-use information stuck in the middle. Practical fix: keep the number of retrieved chunks reasonable (don't stuff 30 chunks in), and put the most important chunk first or last.

9.7 RAG Evaluation (RAGAS-style metrics)

Metric	Question it answers
Context Precision	Of the chunks retrieved, how many were actually relevant?
Context Recall	Of all the relevant chunks that exist, how many did we actually retrieve?
Faithfulness	Does the generated answer actually stick to the retrieved context, or did it hallucinate beyond it?
Answer Relevancy	Does the final answer actually address the user's question?

9.8 Cost and Latency Considerations

- Embedding calls and LLM calls both cost money per token — batch embeddings where possible.
- Smaller, cheaper embedding models are often "good enough" for retrieval — you don't always need the biggest model.
- Re-ranking adds an extra network call and latency — only add it once you've measured that plain vector search isn't

good enough.

- Cache embeddings for documents that don't change.

10. RAG vs Fine-tuning vs Long Context

Approach	What it changes	Strength	Weakness
RAG	What information the model sees, per request	Easy to update knowledge, can cite sources, no retraining	Retrieval can miss relevant chunks; adds pipeline complexity
Fine-tuning	The model's internal weights/behavior	Good for tone, format, specialized skills	Expensive, slow to update, doesn't reliably "store" new facts
Long context (just paste everything in the prompt)	Nothing — brute force, no search needed	Simple if data is small	Expensive at scale, slower responses, "lost in the middle" issues

In real systems, these are often combined: a fine-tuned model that also uses RAG for facts is common.

11. Common Mistakes and How to Avoid Them

Mistake	Fix
Chunk size too large	Reduce size; one chunk should cover roughly one idea
No chunk overlap	Add 10-20% overlap so sentences aren't cut off mid-thought
Different embedding models for ingestion vs query	Always use the exact same model for both
No metadata stored	Always store source, page, date at ingestion time
top-k too small (e.g. 1)	Retrieve more (5-10), then optionally re-rank down to the best few
top-k too large (e.g. 50)	Causes "lost in the middle"; trim with re-ranking or MMR
No fallback when nothing relevant is found	Instruct the LLM to say "I don't know" instead of guessing
Never re-indexing updated documents	Re-run ingestion whenever a source document changes
High LLM temperature	Use a low temperature (0 - 0.3) for factual, grounded answers

12. Interview Questions (Basic to Advanced)

Basic / Conceptual

Q1. What is RAG, in one sentence?

RAG is a technique where an LLM's answer is grounded by first retrieving relevant pieces of external data and feeding them into the prompt as context, instead of relying only on what the model memorized during training.

Q2. Why do we need RAG when LLMs are already very powerful?

Because LLMs have a knowledge cutoff, cannot access private/internal data, have limited context windows, and can hallucinate when they don't actually know an answer. RAG gives the model fresh, specific, and verifiable information at answer time.

Q3. What is an embedding?

A numeric vector that represents the meaning of a piece of text, produced by an embedding model, such that texts with similar meaning end up with vectors that are close together.

Q4. Why do we chunk documents instead of embedding the whole document at once?

Embedding models and LLMs have token limits, and large chunks mix multiple topics together, which weakens the precision of similarity search. Smaller, focused chunks give more accurate retrieval.

Q5. What is chunk overlap and why is it used?

It is a small repeated section of text between two consecutive chunks, used so that information or a sentence sitting right at the chunk boundary isn't lost or split awkwardly.

Intermediate

Q6. What is a vector database, and how is it different from MongoDB or MySQL?

A vector database is optimized to store high-dimensional vectors and quickly find the nearest vectors to a given query vector (similarity search), using ANN algorithms like HNSW. Traditional databases are optimized for exact-match or range queries on structured fields, not similarity search over vectors. (Note: MongoDB Atlas now supports vector search as an added capability.)

Q7. Explain cosine similarity and why it's commonly used for text embeddings.

Cosine similarity measures the angle between two vectors, ignoring their magnitude. For text embeddings, the direction of the vector captures meaning better than its length, so cosine similarity is a reliable way to compare how semantically similar two pieces of text are.

Q8. What is HNSW and why is it used in vector search?

HNSW (Hierarchical Navigable Small World) is an approximate nearest neighbor algorithm that builds a multi-layer graph of vectors. Searches start at a sparse top layer for fast rough navigation, then move into denser layers for precise results, making search far faster than checking every vector (brute force) while staying nearly as accurate.

Q9. What is hybrid search and why would you need it?

Hybrid search combines vector similarity search (meaning-based) with keyword/BM25 search (exact term-based). It's needed because pure vector search can miss exact matches like product codes, IDs, or rare names, which keyword search catches reliably.

Q10. What is the difference between retrieval and generation in RAG?

Retrieval is the search step: finding the most relevant chunks from the vector database for a given query. Generation is the step where the LLM reads those retrieved chunks plus the user's question and writes the final answer.

Q11. Why must you use the same embedding model for both ingestion and query time?

Different embedding models place text in different, incompatible vector spaces. A query vector from one model compared against document vectors from a different model produces meaningless similarity scores, even if both texts mean the same thing.

Q12. What is top-k in retrieval, and how do you choose its value?

Top-k is the number of most similar chunks returned by the search. Too small risks missing relevant information; too large adds noise and can dilute the LLM's focus. A common starting point is 3-10, tuned based on chunk size and evaluation results.

Q13. How do you prevent an LLM from hallucinating in a RAG system?

Use a system prompt that explicitly restricts the model to answer only from the given context, instruct it to say it doesn't know when the context lacks the answer, use a low temperature, and evaluate faithfulness so answers stay grounded in retrieved sources.

Advanced

Q14. What is re-ranking, and why add it after vector search?

Re-ranking uses a more accurate (but slower) model, often a cross-encoder, to re-score a larger initial set of retrieved chunks and pick the truly best few. It's added because fast vector search (using ANN) trades some accuracy for speed, so re-ranking recovers precision on a smaller candidate set.

Q15. What is MMR (Maximal Marginal Relevance) and what problem does it solve?

MMR selects chunks that are both relevant to the query and different from each other, preventing the top-k results from being several near-duplicate chunks that all say the same thing, which would waste context space.

Q16. What is HyDE and when would you use it?

HyDE (Hypothetical Document Embeddings) asks the LLM to first generate a hypothetical answer to the query, then embeds that hypothetical answer (instead of the short query) to perform the search. It helps when the query's wording/style differs a lot from how the answer is phrased in the source documents.

Q17. What is the "lost in the middle" problem?

LLMs tend to pay less attention to information placed in the middle of a long prompt compared to the beginning or end. In RAG, this means stuffing too many retrieved chunks into context can cause the model to under-use some of them, even if they're relevant.

Q18. How would you evaluate whether your RAG system is actually good?

Using metrics such as context precision (are retrieved chunks relevant), context recall (did we retrieve all relevant chunks that exist), faithfulness (does the answer stick to the retrieved context), and answer relevancy (does the answer address the question), often measured with frameworks like RAGAS, alongside human review on a sample set.

Q19. What is Agentic RAG?

An approach where the LLM acts as an agent that can decide, in a loop, whether to retrieve again with a different query, call external tools, or stop and answer — rather than always doing a single fixed retrieve-then-generate pass.

Q20. How is GraphRAG different from standard vector-based RAG?

Standard RAG retrieves isolated text chunks based on vector similarity. GraphRAG builds a knowledge graph of entities and their relationships from the documents, and retrieval can traverse those relationships, which works better for multi-hop questions that connect several facts together.

Q21. Your RAG chatbot gives wrong answers even though the correct information exists in the document. How would you debug it?

Check, step by step: Is the document actually ingested and chunked correctly? Is the relevant chunk being retrieved at all (inspect top-k results directly)? If retrieval is fine but generation is wrong, the prompt or system instructions may need tightening, or the temperature may be too high. If retrieval itself is missing the chunk, consider adjusting chunk size/overlap, trying hybrid search, or adding re-ranking.

Q22. How would you design RAG for a system with millions of documents and strict low-latency requirements?

Use a managed, horizontally scalable vector database with an ANN index like HNSW, keep embedding dimensions reasonably small for speed, use metadata filters to narrow the search space early, add a semantic cache for repeated questions, and consider a lightweight first-pass retrieval followed by re-ranking only on a small candidate set to balance speed and accuracy.

Q23. Can RAG and fine-tuning be used together? Give an example.

Yes. For example, fine-tune a model to respond in a specific tone or format (like a company's support style), while still using RAG to inject up-to-date factual information (like current pricing or policies) that the fine-tuned model alone could never reliably memorize.

Interview tip: Be ready to explain trade-offs, not just definitions. Interviewers often ask "why this and not that" — for example, why HNSW over brute-force k-NN, or why hybrid search over pure vector search. Always answer with the problem it solves.

13. Final Cheat Sheet

Stage	What happens	Tools (Node.js / MERN friendly)
Load	Get raw text out of source files	pdf-parse, mammoth, cheerio
Chunk	Split text into small overlapping pieces	Custom splitter, LangChain.js RecursiveCharacterTextSplitter
Embed	Convert text into vectors	OpenAI / Cohere / Gemini embeddings API, sentence-transformers
Store	Save vector + text + metadata	MongoDB Atlas Vector Search, Pinecone, Chroma, Qdrant
Retrieve	Find top-k similar chunks for a query	\$vectorSearch (Mongo), index.query (Pinecone)
Improve retrieval	Hybrid search, MMR, re-ranking, filtering	Cohere Rerank, custom BM25 + vector merge
Generate	Build prompt with context, call the LLM	OpenAI / Claude / Gemini chat completion APIs
Evaluate	Check faithfulness, relevancy, precision, recall	RAGAS framework, manual review

One paragraph summary: RAG turns an LLM from a "closed-book" guesser into an "open-book" researcher. We chop our documents into small pieces (chunking), turn each piece into a meaning-vector (embedding), and store them in a fast searchable database (vector DB). When a user asks something, we turn their question into a vector too, find the closest matching pieces (retrieval), hand those pieces to the LLM as reference material, and the LLM writes a grounded answer (generation). Everything you build on top — hybrid search, re-ranking, MMR, agentic loops — exists purely to make that core loop more accurate, more relevant, and less prone to hallucination.

End of guide. Build it step by step: first get a basic ingestion + retrieval + generation loop working end-to-end, then layer in hybrid search, re-ranking, and evaluation once the basics are solid.